

Overview of Memory Management

Relocation, Protection, Sharing, Partitioning and Dynamic Storage Allocation

Prashant Jadav

Fundamentals of Operating System Design (FOSD)

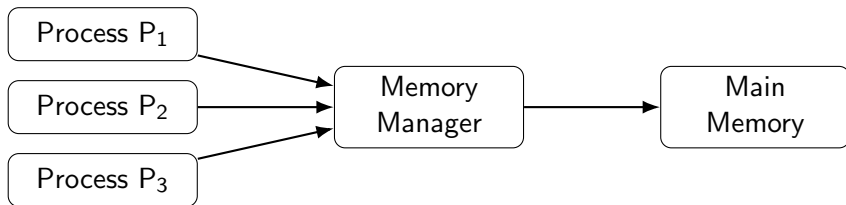
Learning Objectives

- Explain the role of memory management in an operating system.
- Distinguish logical and physical addresses.
- Explain relocation, protection and controlled memory sharing.
- Describe logical and physical organization of memory.
- Explain fixed and dynamic partitioning.
- Distinguish internal and external fragmentation.
- Explain compaction and its limitations.
- Apply First Fit, Best Fit, Worst Fit and Next Fit.
- Compare dynamic storage allocation algorithms using numerical examples.

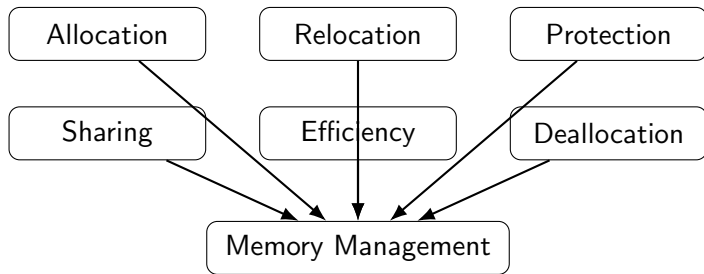
Why Memory Management?

Main memory is limited and shared by processes. The OS must decide:

- which process receives memory;
- where the process is placed;
- how logical addresses are translated;
- how one process is protected from another;
- how memory can be shared safely;
- how free memory is reused efficiently.



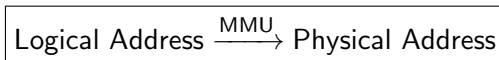
Major Functions of Memory Management



Logical Address and Physical Address

Logical address: address generated by the CPU and used by the process.

Physical address: actual location in main memory.



Relocation

Relocation means translating a process address so that the process can execute at a different physical location.

For a simple base-register model:

$$PA = Base + LA$$

Example:

$$Base = 10000, \quad LA = 250$$

$$PA = 10000 + 250 = 10250$$

If the same process is loaded at base 30000:

$$PA = 30000 + 250 = 30250$$

Key idea

The logical address used by the program remains the same; its physical location can change.

Protection

Memory protection prevents illegal access to memory belonging to the OS or another process.

Typical requirements:

- A user process cannot overwrite another process.
- User code cannot modify protected kernel memory.
- Read-only regions cannot be written.
- Invalid addresses generate a fault or trap.

Simple base-limit protection:

$$0 \leq LA < Limit$$

If valid: $PA = Base + LA$

Example

$Base = 5000$, $Limit = 2000$. For $LA = 1500$:

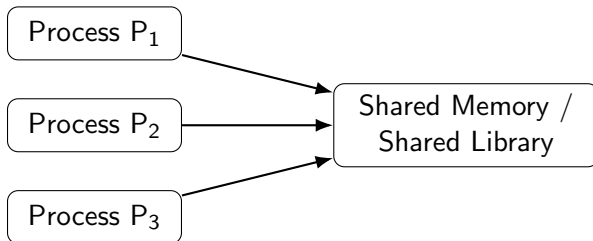
$$PA = 5000 + 1500 = 6500$$

For $LA = 2200$, access is invalid.

Memory Sharing

Memory sharing allows multiple processes to access common memory when permitted. e.g.

- Shared libraries
- Shared-memory IPC
- Read-only program code
- Common data structures



Important

Shared writable memory requires synchronization to prevent race conditions.

Logical Organization of a Process

A process address space can be logically divided into regions.

Stack
Heap
Data
Text / Code

Typical regions:

- Code/text
- Data
- Heap
- Stack

The process sees a logical address space; the OS and hardware map it to physical memory.

Physical Organization of Main Memory

Physical memory is a sequence of addressable locations.

OS / Kernel
Process P_1
Free Space
Process P_2
Free Space

The OS tracks allocated regions and free regions.

Two Classical Contiguous Allocation Approaches

Fixed Partitioning

Memory is divided into fixed-size or predetermined partitions.

$$\text{Memory} \rightarrow P_1 \mid P_2 \mid P_3 \mid \dots$$

Dynamic Partitioning

Partitions are created according to process requests.

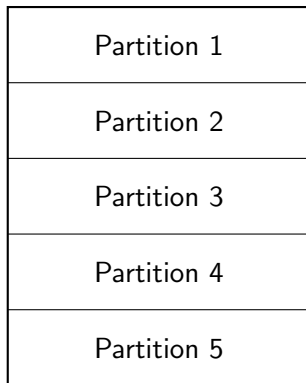
$$\text{Memory} \rightarrow P_1 \mid \text{Free} \mid P_2 \mid \text{Free} \mid P_3$$

Difference

Fixed partitioning fixes boundaries in advance; dynamic partitioning creates boundaries according to allocation requests.

Fixed Partitioning

Main memory is divided into a fixed number of partitions.



If a process is smaller than its assigned partition, the unused space is inside the allocated partition:

Internal Fragmentation

Fixed Partitioning: Numerical Example

Suppose:

$$\text{Memory} = 16 \text{ MB}$$

Four equal partitions:

$$4, 4, 4, 4 \text{ MB}$$

Processes:

$$P_1 = 3 \text{ MB}, \quad P_2 = 2 \text{ MB}, \quad P_3 = 4 \text{ MB}$$

Internal fragmentation:

$$P_1 : 4 - 3 = 1 \text{ MB}$$

$$P_2 : 4 - 2 = 2 \text{ MB}$$

$$P_3 : 4 - 4 = 0$$

Total:

$$3 \text{ MB}$$

Advantages and Disadvantages of Fixed Partitioning

Advantages

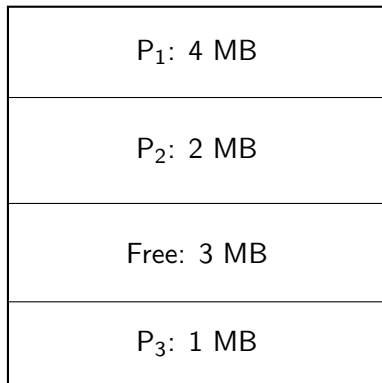
- Simple implementation
- Easy bookkeeping
- Low allocation overhead
- Predictable structure

Disadvantages

- Internal fragmentation
- Limited number of processes
- Poor flexibility for variable sizes
- Partition size may be poorly matched to workload

Dynamic Partitioning

Memory is allocated according to the requested process size.



Dynamic partitioning is flexible but can create:

External Fragmentation

External Fragmentation

External fragmentation occurs when enough total free memory exists but it is divided into separate holes.



Total free:

$$2 + 3 = 5 \text{ MB}$$

A request for 4 MB cannot be satisfied if no single contiguous 4 MB hole exists.

Key point

Total free memory and the largest contiguous free block are different quantities.

Internal vs. External Fragmentation

Feature	Internal	External
Location	Inside allocated block	Between allocated blocks
Typical cause	Fixed-size allocation	Variable-size contiguous allocation
Unused memory	Within an allocated region	Separate free holes
Common example	Fixed partitions	Dynamic partitions
Possible solution	Better partition sizing	Compaction / non-contiguous allocation

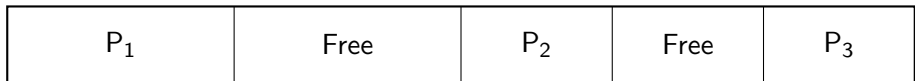
Internal = waste inside

External = holes outside

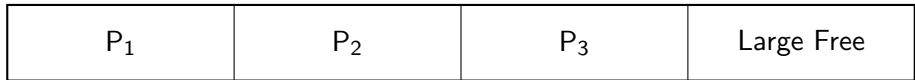
Compaction

Compaction moves allocated processes so that separated free holes form one large contiguous block.

Before:



After:



Cost

Compaction requires moving processes and updating relocation information. It consumes CPU time and memory bandwidth.

When Is Compaction Useful?

Free holes:

2, 4, 3 MB

Total free:

$$2 + 4 + 3 = \boxed{9 \text{ MB}}$$

Request:

8 MB

Without compaction, no single 8 MB hole exists.

After compaction:

$\boxed{9 \text{ MB contiguous}}$

Therefore the 8 MB request can be satisfied.

Important

Compaction changes the distribution of free memory; it does not create additional physical memory.

Dynamic Storage Allocation

Suppose free holes are:

100, 500, 200, 300, 600 KB

A process requests:

212 KB

Possible strategies:

- First Fit
- Best Fit
- Worst Fit
- Next Fit

The selected hole changes the future memory layout.

First Fit

Rule: allocate the first hole encountered that is large enough.

Free holes:

100, 500, 200, 300, 600

Request:

212 KB

First suitable hole:

500 KB

Remaining:

$$500 - 212 = \boxed{288 \text{ KB}}$$

New holes:

100, 288, 200, 300, 600

Property

Search stops when the first suitable hole is found.

Best Fit

Rule: allocate the smallest hole that is large enough.

Suitable holes:

500, 300, 600

For a 212 KB request, the smallest suitable hole is:

300 KB

Remaining:

$$300 - 212 = \boxed{88 \text{ KB}}$$

New holes:

100, 500, 200, 88, 600

Important

Best Fit means the smallest *suitable* hole, not simply the smallest hole.

Worst Fit

Rule: allocate from the largest available suitable hole.

Largest hole:

600 KB

Request:

212 KB

Remaining:

$$600 - 212 = 388 \text{ KB}$$

New holes:

100, 500, 200, 300, 388

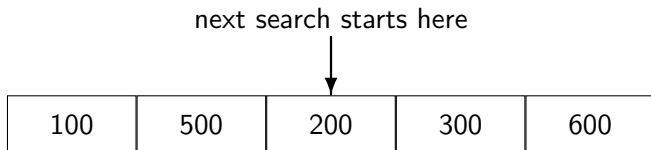
Idea

Leave a relatively large remainder that may satisfy another future request.

Next Fit

Next Fit is similar to First Fit, but the next search begins from the position where the previous search ended.

- Maintain a pointer into the free list.
- Start the next search from that pointer.
- Continue circularly if necessary.



Difference

First Fit starts from the beginning; Next Fit starts from the previous search position.

Comparison of Dynamic Allocation Algorithms

Algorithm	Selection Rule	Main Advantage	Main Concern
First Fit	First adequate hole	Simple and usually fast	Depends on list order
Best Fit	Smallest adequate hole	Small immediate leftover	Can create tiny holes
Worst Fit	Largest adequate hole	Leaves large remainder	May waste large blocks
Next Fit	First adequate hole from previous position	Avoids repeated scan from start	May skip useful earlier holes

Worked Example: One Request

Free holes:

100, 500, 200, 300, 600 KB

Request:

212 KB

Algorithm	Selected Hole	Remaining
First Fit	500 KB	288 KB
Best Fit	300 KB	88 KB
Worst Fit	600 KB	388 KB
Next Fit	Depends on pointer	Depends on pointer

Next Fit

The initial pointer position must be specified before a unique numerical answer can be obtained.

Worked Example: Multiple Requests

Initial holes:

100, 500, 200, 300, 600 KB

Requests:

212, 417, 112, 426 KB

Using First Fit:

212 : 500 $\rightarrow$ 288

417 : 600 $\rightarrow$ 183

112 : 288 $\rightarrow$ 176

Worked Example: Multiple Requests

Current holes:

100, 176, 200, 300, 183

For 426 KB:

Allocation fails

because:

Largest Hole = 300 KB < 426 KB

Why Does Allocation Fail?

From the previous example:

$$\textit{Total Free} = 100 + 176 + 200 + 300 + 183 = 959 \text{ KB}$$

But:

$$\textit{Largest Free Block} = 300 \text{ KB}$$

Request:

$$426 \text{ KB}$$

Therefore:

$$426 > 300$$

Contiguous allocation fails

This is an example of external fragmentation.

Key insight

A system may have enough total free memory but still be unable to satisfy a large contiguous request.

Advantages and Disadvantages of Dynamic Partitioning

Advantages

- Flexible partition sizes
- Lower internal fragmentation
- Suitable for variable process sizes
- Allocation according to demand

Disadvantages

- External fragmentation
- More bookkeeping
- Allocation/deallocation overhead
- Compaction may be required
- Strategy affects future allocations

Fixed vs. Dynamic Partitioning

Feature	Fixed Partitioning	Dynamic Partitioning
Partition size	Predetermined	Based on request
Flexibility	Low	High
Main fragmentation	Internal	External
Bookkeeping	Simple	More complex
Compaction	Usually unnecessary	May be required
Variable process sizes	Less suitable	More suitable

Real-World Analogy

Fixed partitioning

A parking lot has fixed-size spaces. A small car parked in a large space leaves unused space inside that assigned region.

Dynamic partitioning

Parking spaces are created according to vehicle size. When vehicles leave, separated empty spaces can appear.

Compaction

Rearranging vehicles to create one large empty region is analogous to memory compaction.

Design Trade-offs

Memory management balances:

- memory utilization,
- allocation speed,
- low fragmentation,
- implementation complexity,
- protection,
- relocation cost.

Utilization + Speed + Protection + Flexibility + Low Overhead

Key insight

There is no universally optimal allocation strategy for every workload.

Common Mistakes

- Confusing logical and physical addresses.
- Confusing internal and external fragmentation.
- Assuming total free memory is enough for a contiguous request.
- Choosing the smallest hole for Best Fit even when it is too small.
- Choosing the largest hole for Worst Fit without checking suitability.
- Treating Next Fit as identical to First Fit.
- Reusing the original hole list for subsequent requests.
- Ignoring the cost of compaction.

Exam tip

For allocation problems, update the free-hole list after every request.

How to Solve Dynamic Storage Allocation Problems

- ① Write the current free-hole list.
- ② Write the requested size.
- ③ Identify suitable holes.
- ④ Apply the required rule.
- ⑤ Allocate the process.
- ⑥ Calculate the remaining hole.
- ⑦ Update the free list.
- ⑧ Repeat for the next request.

First Fit: first suitable hole

Best Fit: smallest suitable hole

Worst Fit: largest suitable hole

Next Fit: first suitable hole from current pointer

HOT Question 1: Allocation Strategy

Free holes:

100, 500, 200, 300, 600 KB

Requests:

212, 417, 112, 426 KB

- 1 Solve using First Fit.
- 2 Solve using Best Fit.
- 3 Solve using Worst Fit.
- 4 Determine which requests fail.
- 5 Compare the final free-hole distributions.

HOT Question 2: External Fragmentation

A system has:

12 MB free

distributed as:

2, 4, 1, 5 MB

A process requests:

8 MB

- 1 Can contiguous allocation satisfy the request?
- 2 What is the largest free hole?
- 3 Why is total free memory insufficient?
- 4 What does compaction change?
- 5 What is the cost of compaction?

HOT Question 3: Algorithm Selection

A system receives many small memory requests and occasional large requests.

Compare:

First Fit, Best Fit, Worst Fit, Next Fit

Discuss:

- allocation search cost,
- fragmentation,
- future allocation success,
- sensitivity to request order,
- workload suitability.

Higher-order task

Design an experiment to measure allocation success and external fragmentation for each strategy.

HOT Question 4: Protection and Relocation

A process has:

$$Base = 12000, \quad Limit = 4000$$

For:

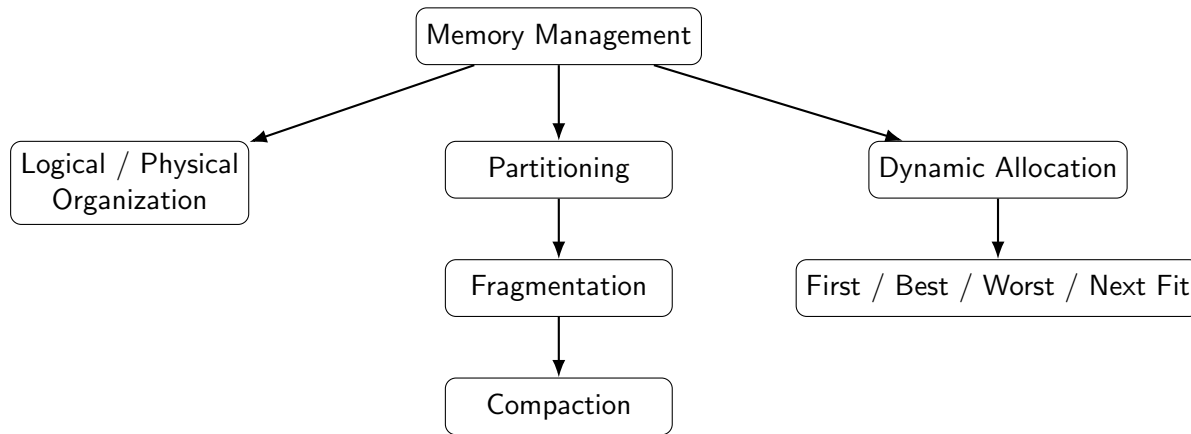
$$LA = 500, \quad 3500, \quad 4000, \quad 4500$$

- 1 Determine which references are valid.
- 2 Calculate physical addresses for valid references.
- 3 Explain the role of the limit.
- 4 Explain how relocation allows the process to move to another physical address.

Summary

- Memory management controls allocation, protection, relocation and sharing.
- Logical addresses are translated to physical addresses.
- Relocation permits a process to execute at different physical locations.
- Protection prevents illegal memory access.
- Sharing permits controlled common-memory access.
- Fixed partitioning is simple but causes internal fragmentation.
- Dynamic partitioning is flexible but can cause external fragmentation.
- Compaction combines separated free holes but has a cost.
- First Fit chooses the first suitable hole.
- Best Fit chooses the smallest suitable hole.
- Worst Fit chooses the largest suitable hole.
- Next Fit continues from the previous search position.

Final Conceptual Map



Efficient memory management balances utilization, speed, protection and flexibility